

Introdução a Algoritmos

Aula 4 – Lógica, switch e Legibilidade

Raoni F. S. Teixeira · 2s/2026

Objetivos desta aula

Ao final desta aula, você deverá ser capaz de:

1. Combinar condições com os operadores `&&` , `||` e `!` para substituir ifs aninhados por uma única expressão equivalente;
2. Implementar um menu de opções usando switch, incluindo o tratamento do caso default e o uso correto de break;
3. Identificar o efeito do fall-through num switch sem break e determinar se ele é intencional ou um bug;
4. Reescrever um trecho de código ilegível aplicando os quatro pilares da legibilidade: nomes descritivos, indentação, comentários úteis e documentação de cabeçalho.

1 Combinando condições: operadores lógicos

Na aula anterior, cada condição testava **uma única relação** — por exemplo, `x > 0` ou `n == 42`. Muitos problemas reais exigem condições mais complexas: “o usuário é maior de idade **e** tem carteira de motorista” ou “o produto está em promoção **ou** o cliente tem cupom de desconto”.

Para expressar essas condições compostas, C oferece os **operadores lógicos**.

Definição — Expressão Lógica

Uma expressão lógica combina duas ou mais expressões booleanas por meio de operadores lógicos (E, OU, NÃO) e produz um resultado booleano (verdadeiro ou falso).

Os três operadores lógicos de C são:

Operador	Nome	Resultado
<code>&&</code>	E lógico (AND)	Verdadeiro somente se ambas as expressões forem verdadeiras
<code> </code>	OU lógico (OR)	Verdadeiro se pelo menos uma das expressões for verdadeira
<code>!</code>	NÃO lógico (NOT)	Inverte o valor: transforma verdadeiro em falso e vice-versa

1.1 Tabelas-verdade

As tabelas-verdade mostram o resultado de cada operador para todas as combinações de entrada:

A	B	A && B
V	V	V
V	F	F
F	V	F
F	F	F

Operador AND

A	B	A B
V	V	V
V	F	V
F	V	V
F	F	F

Operador OR

A	!A
V	F
F	V

Operador NOT

1.2 Exemplos práticos

```
int idade = 20, tem_habilitacao = 1;

/* Pode dirigir? Precisa ter 18+ E habilitação */
if (idade >= 18 && tem_habilitacao) {
    printf("Pode dirigir.\n");
}

/* Tem desconto? Sócio OU compra acima de R$200 */
float total = 150.0;
int eh_socio = 0;
if (eh_socio || total > 200.0) {
    printf("Desconto aplicado.\n");
}

/* Número fora do intervalo [0, 100] */
int n = 110;
if (n < 0 || n > 100) {
    printf("Valor inválido.\n");
}
```

1.3 Avaliação em curto-circuito

C avalia expressões lógicas da esquerda para a direita e **para assim que o resultado for determinado**:

- Em A && B: se A for falso, B **não é avaliado** — o resultado já é falso.
- Em A || B: se A for verdadeiro, B **não é avaliado** — o resultado já é verdadeiro.

Esse comportamento — chamado de **avaliação em curto-circuito** — é importante quando a segunda expressão tem efeitos colaterais ou pode causar erro:

```
/* Seguro: só divide se denominador != 0 */
if (b != 0 && a / b > 2) {
    printf("Quociente maior que 2.\n");
}
```

Se b for zero, a condição b != 0 é falsa e a divisão a / b **nunca é executada**, evitando uma divisão por zero.

1.4 Simplificações úteis

As negações de expressões relacionais têm formas equivalentes mais simples:

Expressão original	Forma equivalente
!(a == b)	a != b
!(a != b)	a == b
!(a > b)	a <= b
!(a < b)	a >= b
!(a >= b)	a < b
!(a <= b)	a > b

1.5 Simplificando ifs aninhados

Operadores lógicos permitem substituir ifs aninhados por uma única condição, tornando o código mais claro:

```
/* Versão com ifs aninhados – difícil de ler */
if (cond1) {
    if (cond2) {
        if (!cond3) {
            instrucao;
        }
    }
}

/* Versão equivalente com operadores lógicos – clara */
if (cond1 && cond2 && !cond3) {
    instrucao;
}
```

2 O comando switch: decisões sobre um único valor

A estrutura if-else-if é versátil, mas quando todos os casos testam o **mesmo valor** contra constantes diferentes, ela pode ser substituída pelo comando switch, que é mais legível e, em alguns compiladores, mais eficiente.

```
switch (expressao) {
    case valor1:
        instrucoes;
        break;
    case valor2:
        instrucoes;
        break;
    default:
        instrucoes; /* executado se nenhum case corresponder */
}
```

Exemplo – Menu de opções

```
int opcao;
printf("1-Novo 2-Abrir 3-Salvar 0-Sair\n");
```

```
scanf("%d", &opcao);

switch (opcao) {
    case 1:
        printf("Criando novo arquivo...\n");
        break;
    case 2:
        printf("Abrindo arquivo...\n");
        break;
    case 3:
        printf("Salvando arquivo...\n");
        break;
    case 0:
        printf("Saindo...\n");
        break;
    default:
        printf("Opção inválida.\n");
}
```

2.1 O papel do break — e o que acontece sem ele

O comando `break` encerra a execução do `switch` e pula para a instrução seguinte. Sem ele, a execução **cai** para o próximo `case` — comportamento chamado de **fall-through**.

```
switch (x) {
    case 1:
        printf("um\n");
        /* sem break — execução continua! */
    case 2:
        printf("dois\n");
        break;
    case 3:
        printf("três\n");
        break;
}
/* Para x == 1, imprime "um" E "dois" */
```

O `fall-through` é raramente intencional em programas iniciantes e quase sempre representa um bug. Sempre inclua `break` ao final de cada `case`, a menos que você tenha uma razão explícita para não fazê-lo (e, nesse caso, documente com um comentário).

Atenção

O `switch` só aceita valores inteiros (incluindo `char`). Não é possível usar `switch` com `float`, `double` ou strings — para esses casos, use `if-else-if`.

2.2 Quando usar switch vs. if-else-if

Use <code>switch</code> quando...	Use <code>if-else-if</code> quando...
Todos os casos testam o mesmo valor inteiro contra constantes	As condições envolvem intervalos ou expressões diferentes

O número de casos é grande e bem definido (menus, estados, códigos)	As condições envolvem float, double ou strings
Você quer aproveitar o fall-through intencionalmente	As condições são complexas ou combinadas com && e

3 Legibilidade de código

Até aqui falamos sobre **o que** o código faz. Nesta seção, discutimos **como** ele deve ser escrito para que outros — e você mesmo, no futuro — possam entendê-lo facilmente.

Definição — Legibilidade

Legibilidade é a qualidade de um código que permite seu entendimento rápido e preciso por qualquer programador familiarizado com a linguagem, sem necessidade de explicações adicionais.

Um código é escrito uma vez, mas lido e mantido muitas vezes. Em projetos reais, mais de 70% do tempo de desenvolvimento é gasto em **manutenção** — corrigindo bugs, adicionando funcionalidades, adaptando o código a novos requisitos. Código ilegível torna esse trabalho exponencialmente mais difícil.

3.1 Os quatro pilares da legibilidade

3.1.1 1. Nomes descritivos

Nomes de variáveis e funções devem comunicar sua finalidade sem exigir comentário adicional.

```
/* Ruim: o leitor não sabe o que n1, n2 e r representam */
float n1, n2, r;
scanf("%f %f", &n1, &n2);
r = (n1 * 2 + n2 * 3) / 5;

/* Bom: o código se auto-documenta */
float nota_prova, nota_trabalho, media_final;
scanf("%f %f", &nota_prova, &nota_trabalho);
media_final = (nota_prova * 2 + nota_trabalho * 3) / 5;
```

3.1.2 2. Indentação consistente

A indentação revela visualmente a estrutura lógica do programa — quais instruções estão dentro de quais blocos.

```
/* Ruim: sem indentação, estrutura oculta */
if(n==1){printf("Unidade");}else if(n>=0){if(n%3)
printf("Deixa resto");else printf("Não deixa");}else{printf("Negativo");}

/* Bom: estrutura clara à primeira vista */
if (n == 1) {
    printf("Unidade\n");
} else if (n >= 0) {
    if (n % 3) {
        printf("Deixa resto\n");
    }
}
```

```
    } else {  
        printf("Não deixa\n");  
    }  
} else {  
    printf("Negativo\n");  
}
```

3.1.3 3. Comentários que acrescentam, não que repetem

Um comentário ruim repete o que o código já diz. Um comentário bom explica o **porquê** de uma decisão que não é óbvia.

```
/* Ruim: o comentário não acrescenta nada */  
x = x + 1; /* soma 1 a x */  
  
/* Bom: explica a razão da operação */  
total_lidos++; /* conta a leitura mesmo que o valor seja inválido */  
  
/* Bom: documenta uma fórmula não-óbvia */  
/* Cálculo do índice de massa corporal: peso(kg) / altura(m)^2 */  
imc = peso / (altura * altura);
```

3.1.4 4. Documentação do programa

Todo programa deve ter um cabeçalho que responde três perguntas: o que faz, quem escreveu e quando. Para funções não-triviais, adicionamos também as entradas, saídas e pré-condições.

```
/*  
 * Descrição: Lê o raio de uma esfera e imprime seu volume.  
 * Entrada:   raio (float, em metros)  
 * Saída:     volume (em metros cúbicos), com 4 casas decimais  
 * Pré-cond.: raio > 0  
 * Autor:     Fulano de Tal  
 * Data:      março de 2025  
 */
```

3.2 Exemplo completo: antes e depois

Antes — código funcional, mas ilegível:

```
#include<stdio.h>  
#include<math.h>  
int main(){float x,y,z;scanf("%f",&x);scanf("%f",&y);  
z=x*x+y*y;z=sqrt(z);printf("%f\n",z);}
```

Depois — mesmo comportamento, totalmente legível:

```
#include <stdio.h>  
#include <math.h>  
  
/*  
 * Descrição: Calcula a hipotenusa de um triângulo retângulo.  
 * Entrada:   comprimentos dos dois catetos (reais positivos)  
 * Saída:     comprimento da hipotenusa  
 * Pré-cond.: catetos > 0  
 */  
int main() {
```

```
float cateto_a, cateto_b, hipotenusa;

/* Leitura dos catetos */
printf("Cateto a: ");
scanf("%f", &cateto_a);
printf("Cateto b: ");
scanf("%f", &cateto_b);

/* Teorema de Pitágoras:  $h^2 = a^2 + b^2$  */
hipotenusa = sqrt(cateto_a * cateto_a + cateto_b * cateto_b);

printf("Hipotenusa: %f\n", hipotenusa);
return 0;
}
```

Legibilidade não é um luxo para quando o código “já estiver funcionando” — é uma disciplina que se pratica desde a primeira linha. Código legível é mais fácil de depurar, de corrigir e de expandir. Adote os quatro pilares desde hoje.

4 Exercícios

1. Reescreva os programas do exercício 1 da aula anterior usando operadores lógicos para simplificar os ifs aninhados.
2. Escreva um programa com switch que leia um número de 1 a 7 e imprima o nome do dia da semana correspondente (1 = segunda, ..., 7 = domingo). Para valores fora do intervalo, exiba “Dia inválido”.
3. Escreva um programa que leia duas datas (dia, mês, ano) e determine qual ocorreu primeiro. Cuide da legibilidade: use nomes descritivos e comente as comparações não-óbvias.
4. Avalie o código abaixo segundo os quatro pilares de legibilidade e reescreva-o:

```
int main(){int a,b,c,d;scanf("%d%d%d%d",&a,&b,&c,&d);
if(a>1900&&a<2100)if(b>=1&&b<=12)if(c>=1&&c<=31)
printf("ok");else printf("nd");else printf("nm");else printf("na");}
```